

SECURITY SPECIFICATION · VERSION 4.0

SaveHatke AI Chatbot

Highest-Practical-Security Baseline

A defense-in-depth, fail-closed production security specification for the SaveHatke AI assistant, its tool-chain, its data access paths and the backend services that hold authority over every sensitive operation.

- AI

= Assistant
- Backend

= Authority
- Database

= Protected Resource
- User

= Untrusted Until Authenticated and Authorized

DOCUMENT ID	SUPERSEDES	STATUS
SH-SEC-CHATBOT-4.0	v3.2 — Ultra-Security Baseline	Production Security Baseline
OWNER	ISSUED	REVIEW CYCLE
Platform Security · SaveHatke	29 August 2026	Quarterly & on material change

This document defines the **strongest practical security controls** for the SaveHatke AI Chatbot. It does not claim that the system is impossible to attack. No system is. It claims that every sensitive operation is independently authenticated, authorized, validated and logged by trusted backend services, that the AI is never the final authority, and that controls are continuously tested, monitored and improved. Where a control cannot establish that an operation is permitted, the operation fails closed.

00 Document Control, Scope & Reading Guide

FRONT MATTER

Version 4.0 is a direct upgrade of the v3.2 Ultra-Security baseline. No control from v3.2 has been removed, relaxed or replaced with a shorter summary. Every previous section is retained and extended with the additional defense-in-depth layers required for production operation of an AI assistant that sits in front of real user accounts, real orders and real money.

WHAT CHANGED FROM V3.2 TO V4.0

AREA	UPGRADE APPLIED IN V4.0	STATUS
Identity & Session	Full session lifecycle: rotation on login and privilege change, revocation, device management, idle and absolute timeouts, step-up and re-authentication, fixation / replay / stolen-token defenses	Expanded
Authorization	Object-level authorization on every private resource, RBAC and ABAC guidance, explicit ban on trusting client-supplied identifiers	Expanded
AI Runtime	New section: AI Runtime Isolation — no database credentials, no shell, no arbitrary SQL, no metadata endpoints, allowlisted short-lived tools	New
Tool-Chain	Mandatory eleven-stage tool-call pipeline, prohibited capability list, call budgets, timeouts and circuit breakers	Expanded
Memory	New section: conversation context and memory security, cross-user memory isolation, memory-poisoning defense, retention limits	New
Prompt Injection	Full attack taxonomy including indirect, multi-turn, multilingual, encoded, invisible-character, tool-result and RAG vectors	Expanded
Exfiltration	New section: authorization-aware retrieval, output redaction, secret scanning, cross-user and tenant isolation	New
Data Layer	New sections: Database Security and Secrets & Key Management, including least privilege, encryption, rotation and emergency revocation	New
Financial	Hardened payment isolation, webhook signature verification, idempotency, velocity checks, approval workflows, financial audit trail	Expanded
Cost & Abuse	New section: AI denial-of-service and cost protection with per-user and global ceilings, budget alerts and emergency shutdown	New
Operations	Monitoring and detection catalogue, fourteen incident-response playbooks, continuous post-launch red-teaming, supply-chain controls	Expanded
Release Gate	Twenty named production gate checks, four standing post-release gates and a formal security sign-off block	Expanded

Scope

The SaveHatke AI Chatbot and every path it touches: chat frontend, chat API, authentication and session layer, AI runtime, tool layer, retrieval and knowledge base, conversation storage, backend business logic, database, payment integrations, file pipeline, outbound fetch, admin surfaces, logging and monitoring.

Out Of Scope

Chatbot visual design, conversational tone, product feature roadmap and non-security UX. This document changes no user-facing interface behaviour except where a security control legitimately requires it, such as step-up authentication prompts or rate-limit messaging.

LANGUAGE POLICY FOR THIS DOCUMENT

No Absolute Guarantees

This baseline deliberately avoids claims such as “100% secure”, “impossible to hack” or “bypass-proof”. Such claims are unverifiable and create false confidence during review. The approved vocabulary is: **strongest practical security controls, defense in depth, production-grade security baseline, continuously tested security and fail-closed architecture**. Anyone extending this document must keep that vocabulary.

Confidential — Internal Security Documentation

SH-SEC-CHATBOT-4.0

Page 2 of 34

—

Contents

25 CONTROL SECTIONS

FOUNDATION			HIGH-RISK SURFACES		
01	Security Doctrine, Threat Model & Trust Boundaries	4	13	Payment & Financial Security	21
IDENTITY & THE AI BOUNDARY			14	File & Image Security	22
02	Access Control & Identity	6	15	Web Fetch, URL & SSRF Security	23
03	AI Runtime Isolation	9	ABUSE, COST & OPERATIONS		
04	Prompt Injection & Jailbreak Protection	10	16	Bot Abuse & Automation Defense	24
05	Tools, Agents & Excessive-Agency Protection	12	17	AI Denial-of-Service & Cost Protection	25
06	AI Memory & Conversation Context Security	14	18	Security Monitoring & Detection	26
07	AI Data Exfiltration Prevention	15	19	Incident Response & Recovery	27
08	RAG & Knowledge Base Security	16	20	Continuous Security Testing	29
09	AI Output Security	17	21	AI & Software Supply-Chain Security	30
PLATFORM & DATA LAYER			22	Administrative Access Security	31
10	Web, API & Infrastructure Hardening	18	CLOSING		
11	Database Security	19	23	Final Security Architecture	32
12	Secrets & Key Management	20	24	Production Release Gate	33
			25	Final Security Principle & Sign-Off	34

HOW TO READ THIS DOCUMENT

AUDIENCE	PRIMARY SECTIONS
Engineering	2–3 (identity, isolation), 4–5 (injection, tools), 9–12 (output, API, database, secrets), 13–15 (payments, files, fetch)
Security Review	1 (threat model), 6–8 (memory, exfiltration, retrieval), 20 (continuous testing), 24 (release gate)
QA & Test	20 (test catalogue), 24 (gate checklist), plus the verification notes inside each control section
Operations	16–19 (abuse, cost, monitoring, incident response), 21–22 (supply chain, admin)
Leadership	Cover doctrine, 1 (threat model), 23 (architecture), 25 (final principle)

Control Status Legend

MUST

 mandatory before production ·

SHOULD

 required unless a documented exception is approved by Platform Security ·

MAY

 recommended where practical ·

VERIFY

 requires an explicit test result recorded in the release gate.

01

Security Doctrine, Threat Model & Trust Boundaries

FOUNDATION

Every control in this document derives from one doctrine. If a future change appears to conflict with the doctrine, the doctrine wins and the change is rejected until redesigned.

AI = Assistant

The model interprets language, explains, drafts, summarises and recommends. It proposes actions. It never decides whether an action is permitted and never performs a privileged operation itself.

Backend = Authority

Trusted server-side services authenticate the caller, authorize the operation, recalculate every value that matters, enforce business rules, commit state and write the audit record.

Database = Protected Resource

The datastore is reachable only through backend services using least-privilege credentials. No client, no browser and no AI runtime ever holds database credentials or issues free-form queries.

User = Untrusted Until Authenticated and Authorized

Identity claims, role claims, ownership claims, prices, totals, identifiers and eligibility statements coming from the client or from chat text are treated as input to be verified, never as fact.

TRUST LEVELS

COMPONENT	TRUST	MEANING IN PRACTICE
Backend business logic	Trusted	Sole authority for authentication, authorization, calculation, state change and audit
Datastore & payment provider	Trusted	Reached only from backend services over authenticated, least-privilege channels
Chat API gateway	Semi-trusted	Enforces transport security, rate limits, schemas and session validation; delegates all decisions to backend logic
AI runtime & model output	Untrusted	Isolated process, no credentials, no direct data access; output is data to be validated, never a command
Retrieved documents, tool results, fetched pages	Untrusted	Content, not instructions; scanned for injection and never permitted to change policy or privileges
Browser, mobile app, frontend code	Untrusted	Fully attacker-controllable; client validation is a usability feature, never a security control
User-supplied text, files, URLs, identifiers	Untrusted	Validated, normalised and authorized on the server before any use

Core Rule

Fail closed. If a security check cannot positively establish that an operation is permitted — because a session is unverifiable, an authorization service is unreachable, a schema does not validate, a limit cannot be evaluated or an ownership record cannot be confirmed — the operation is refused and logged. Absence of a denial is never treated as permission.

01 Threat Actors, Attack Surface & Assumptions

SECTION 01 CONTINUED

THREAT ACTORS & PRIMARY OBJECTIVES

ACTOR	OBJECTIVE	PRINCIPAL CONTROLS
Curious authenticated user	Read another user's orders, wallet or support history	Object-level authorization, authorization-aware retrieval, cross-user isolation (§2, §7)
Prompt-injection attacker	Make the assistant ignore policy, escalate privilege or leak data	Instruction hierarchy, data/instruction separation, tool authorization (§4, §5)
Content-supply attacker	Poison a document, webpage or stored memory to steer future sessions	Provenance checks, injection scanning, poisoned-document detection (§6, §8)
Financial fraudster	Obtain goods, refunds, payouts or wallet credit without valid payment	Server-side pricing and verification, idempotency, approval workflows (§13)
Automation & scraping operator	Bulk-extract catalogue or user data, exhaust AI budget	Layered rate limits, cost ceilings, automation detection (§16, §17)
Infrastructure prober	Reach internal services, cloud metadata or credentials	Egress filtering, SSRF blocklists, secret management (§12, §15)
Account-takeover attacker	Reuse stolen sessions, tokens or credentials	Session rotation, revocation, step-up auth, anomaly detection (§2, §18)
Supply-chain attacker	Compromise a dependency, container image or model integration	Pinning, scanning, SBOM, provider and version review (§21)
Malicious insider or compromised admin	Abuse privileged tooling or bulk data access	Least privilege, role separation, MFA, two-person approval, audit (§22)

ATTACK SURFACE INVENTORY

- Chat message input, including files, links and pasted content
- Conversation history and long-lived memory
- System prompt and instruction hierarchy
- Tool and function-calling interface
- Retrieval / knowledge-base pipeline
- Outbound URL fetch and page summarisation
- File upload, OCR and vision processing
- Chat API and every backend endpoint it reaches
- Session tokens, cookies and device records
- Payment initiation, webhooks and wallet operations
- Support, order and account data paths
- Admin and internal tooling surfaces
- Logs, traces and analytics sinks
- Third-party AI provider and integration boundary
- Build pipeline, dependencies and container images
- Secret storage and credential distribution

STATED ASSUMPTIONS

Any input reaching the model may be attacker-controlled. Any model output may be attacker-influenced. Any client-side check may be removed. Any token may be stolen. Any dependency may be compromised. Controls are therefore layered so that a single failed assumption does not become a breach, and so that the failure of one layer is detected by another.

02 Access Control & Identity

AUTHENTICATION

Governing Rule

Authentication proves identity. Authorization determines what that identity is allowed to access. The two are separate decisions, made by separate code paths, and both are made on the server for every request — including every request that originates inside a chat conversation.

AUTHENTICATION CONTROLS

CONTROL	REQUIREMENT	LEVEL
Server-side verification	Every authenticated route verifies the session server-side before executing logic; no route infers identity from request body content	MUST
Credential storage	Passwords stored with a modern memory-hard hash (Argon2id or bcrypt with tuned cost); never encrypted-reversible, never plain	MUST
Brute-force resistance	Per-account and per-IP throttling, progressive delay, lockout with safe recovery, and constant-time comparison	MUST
Credential-stuffing defense	Breached-password rejection, anomaly scoring on login velocity, and challenge escalation on suspicious attempts	SHOULD
Multi-factor authentication	Available to all users, mandatory for staff and admin accounts, enforced server-side at the session-issuing step	MUST
OTP handling	Short expiry, single use, rate-limited issuance, bound to the requesting context, never echoed back and never written to logs or chat transcripts	MUST
Account recovery	Recovery paths held to the same strength as login; recovery cannot bypass MFA silently, and recovery events force full session revocation	MUST
Uniform error responses	Login and recovery responses do not reveal whether an identifier exists; timing and message shape are uniform	SHOULD
Federated login	Provider tokens validated against the provider, audience and issuer checked, email-verification state confirmed before account linking	MUST
AI exclusion	The assistant never collects, relays, confirms or verifies passwords, OTPs, recovery codes or MFA seeds; authentication is completed only in first-party authenticated UI	MUST

STEP-UP & RE-AUTHENTICATION TRIGGERS

Sensitive operations require a fresh authentication event even inside a valid session. The backend decides when step-up applies; the assistant may only tell the user that it is required.

OPERATION CLASS	REQUIREMENT
Credential & MFA changes	Re-authentication, plus notification to the account's verified contact channels and full session revocation on success
Payout, bank or UPI destination change	Step-up authentication, cool-down period before first payout to the new destination, and manual review above threshold
Refunds, wallet withdrawal, high-value orders	Step-up authentication and backend approval workflow (§13); never completed on the basis of a chat instruction alone
Email, phone or address change	Re-authentication plus verification of the new destination before it becomes usable for recovery
Data export or account deletion	Re-authentication, delay window, and reversible grace period with notification
Any privileged or staff action	Separate admin session, MFA, and re-authentication for critical actions (§22)

02 Secure Session Lifecycle Management

SECTION 02 CONTINUED

A session is the runtime representation of a proven identity. Its issuance, rotation, validation, expiry and revocation are security-critical events and are all controlled server-side.

LIFECYCLE REQUIREMENTS

STAGE	REQUIREMENT	LEVEL
Issuance	Session identifiers are generated server-side from a cryptographically secure random source with sufficient entropy; never derived from user data, never predictable, never sequential	MUST
Rotation after login	A new session identifier is issued on every successful authentication and the pre-authentication identifier is invalidated — the primary defense against session fixation	MUST
Rotation after privilege change	Any change in role, permission set, tenant context, MFA state or elevation level forces re-issuance so no token continues to carry stale authority	MUST
Server-side validation	Every request validates the session against server-side state: existence, expiry, revocation status, bound device record and elevation level. Signature validity alone is not sufficient	MUST
Idle timeout	Sessions expire after a defined period of inactivity; shorter for staff and admin sessions than for normal users	MUST
Absolute lifetime	Every session has a maximum total lifetime after which re-authentication is required regardless of activity; refresh cannot extend a session indefinitely	MUST
Logout	Logout invalidates server-side session state immediately and clears client credentials; a logged-out token is never accepted again	MUST
Revocation	Support for revoking an individual session, all sessions for one account, or a set of sessions matching a compromise indicator, with effect on the next request	MUST
Forced revocation events	Password change, MFA change, recovery completion, suspected compromise, staff role change and account deletion all revoke every existing session	MUST
Device & session management	Users can view active sessions with device, approximate location, first-seen and last-seen data, and can terminate any of them individually or all at once	SHOULD
Concurrent-session anomaly detection	Impossible-travel patterns, rapid device or network changes, and abnormal parallel-session counts raise a security signal and can force step-up or revocation	SHOULD
Replay & stolen-token defense	Tokens bound to session context, refresh-token rotation with reuse detection, and automatic revocation of the whole token family when a rotated token is presented again	MUST

SECURE COOKIE & TOKEN CONFIGURATION

- **HttpOnly** on all session cookies
- **Secure** flag; HTTPS-only transport
- **SameSite** set restrictively for the flow
- Narrow **Path** and explicit **Domain** scope
- No session material in URLs or query strings
- No session material in browser local storage
- Signed and integrity-checked cookie values
- Short-lived access tokens, rotating refresh tokens
- Clock-skew tolerance kept minimal
- Distinct cookie names and scopes per environment
- Separate cookie scope for admin surfaces
- Session-signing secrets rotated on schedule (§12)

Chat-Specific Session Rules

A chat conversation is not an authentication context. Long-running conversations are re-validated on every message: an expired, revoked or downgraded session immediately loses access to private tools and private data even if earlier turns in the same conversation succeeded. The assistant cannot extend, refresh, restore or elevate a session, and it cannot carry authority forward from a previous turn.

02 Authorization & Object-Level Access Control

SECTION 02 CONTINUED

Never Trust Client-Provided Identity

User identifiers, account identifiers, role names, tenant identifiers, permission flags and entitlement claims are never accepted from the request body, query string, headers, chat text or model output. The acting identity is derived exclusively from the validated server-side session, and every resource lookup is scoped to that derived identity.

AUTHORIZATION MODEL

CONTROL	REQUIREMENT	LEVEL
Deny by default	Access is refused unless an explicit policy permits it. New endpoints, new tools and new fields inherit denial, not permission	MUST
Object-level authorization	Every private resource — order, ticket, wallet entry, payout, address, document, conversation, uploaded file — is checked for ownership or explicit grant on every read, write and delete	MUST
Function-level authorization	Every endpoint and every tool declares the permission it requires; possessing a valid session does not imply the right to invoke it	MUST
Role-based access control	Coarse-grained roles (customer, support, finance, admin) define baseline capability sets with no implicit inheritance of higher roles	MUST
Attribute-based rules	Where a role is insufficient, decisions additionally consider resource state, ownership, account standing, verification level and session elevation	SHOULD
Centralised enforcement	Authorization is evaluated in shared server-side policy code, not duplicated ad hoc per handler, so that a fix applies everywhere at once	SHOULD
Query-level scoping	Data access is filtered by owner at the query layer as well as checked after retrieval, so a missing check cannot silently return foreign rows	MUST
Field-level restriction	Responses are built from explicit allowlisted fields; internal columns, cost data, staff notes and system flags are never serialised to a user context	MUST
Tenant isolation	Where multi-tenant or partner data exists, tenant scope is bound to the session and enforced independently of any client-supplied tenant value	MUST

MANDATORY AUTHORIZATION DECISION POINTS

POINT	WHAT IS CHECKED
Chat API entry	Session validity, account standing, rate-limit state, elevation level
Before every tool call	Tool permission, resource ownership, argument scope, operation allowlist (§5)
Before every retrieval	Document and chunk permissions for the acting identity (§8)
Before memory access	Conversation ownership and retention eligibility (§6)
Inside backend business logic	Independent re-check of identity, ownership, state and business rules — never trusting that the caller already checked
Before response assembly	Output filtering and field allowlisting for the acting identity (§7, §9)

Restated

Authentication proves identity. Authorization determines what that identity is allowed to access. A valid session is never sufficient justification for reaching a resource, and no conversational context, prior turn or model assertion can substitute for an authorization decision made by trusted backend code.

Cross-user and IDOR test cases for these controls are catalogued in §20 Continuous Security Testing and recorded in the §24 release gate.

03

AI Runtime Isolation

NEW IN V4.0

The model and its orchestration logic run in a confined runtime whose maximum reachable capability is a small set of narrowly scoped, allowlisted APIs. Isolation is what keeps a successful prompt injection from becoming a platform compromise: the attacker may influence what the assistant *says*, but not what the runtime is *able* to do.

RUNTIME PROHIBITIONS

- ×

No direct database credentials or connections
- ×

No arbitrary or free-form SQL execution
- ×

No arbitrary shell or process execution
- ×

No arbitrary code evaluation at runtime
- ×

No arbitrary browser or client-side code execution
- ×

No unrestricted outbound network access
- ×

No access to server configuration or environment
- ×

No ability to modify deployment or infrastructure
- ×

No filesystem access to secrets, keys or config files
- ×

No access to cloud metadata endpoints
- ×

No access to internal service networks not explicitly allowed
- ×

No ability to create, alter or grant permissions
- ×

No ability to reach other users' data stores
- ×

No ability to write to logs, audit records or monitoring sinks

ISOLATION CONTROLS

CONTROL	REQUIREMENT	LEVEL
Process & network confinement	The AI runtime executes in its own container or sandbox with a dedicated minimal service identity, read-only root filesystem where practical, and default-deny egress (§15)	MUST
No credential inheritance	The runtime holds no database, payment, admin or infrastructure credentials. Provider API keys are held by a separate gateway service, not injected into model context (§12)	MUST
Narrow tool APIs	Tools expose specific business operations with typed parameters — never a generic query, fetch, execute or admin capability	MUST
Explicit allowlist	The set of callable tools is a static server-side allowlist. Tools not on the list cannot be invoked even if named perfectly by the model	MUST
Session-scoped, short-lived grants	Tool credentials and data-access grants are minted per request or per session, expire quickly, and are revoked when the session ends or is revoked	MUST
Identity propagation, not elevation	Tool calls execute strictly as the authenticated end user; the runtime has no service-level identity capable of bypassing user authorization	MUST
Tool output validation	Every tool result is schema-validated and treated as untrusted content before it re-enters model context or reaches the user (§5, §9)	MUST
Resource ceilings	CPU, memory, execution time, concurrency and token budgets are capped so a single conversation cannot exhaust shared capacity (§17)	MUST
Blast-radius review	Each new tool is reviewed for the worst outcome achievable if the model were fully attacker-controlled; unacceptable answers block the tool	MUST

Isolation Rule

Assume the prompt is compromised and ask what remains reachable. If the honest answer includes credentials, foreign user data, configuration, internal networks, arbitrary execution or irreversible financial state, the isolation boundary is wrong and must be narrowed before release.

04 Prompt Injection & Jailbreak Protection

ATTACK TAXONOMY

Governing Rule

Content retrieved from users, files, websites, RAG systems, APIs and tools is DATA, never higher-priority instructions. No text encountered at runtime — regardless of how it is phrased, formatted, encoded, signed or attributed — can raise its own privilege, alter policy, unlock a tool or authorise an operation.

RECOGNISED INJECTION & JAILBREAK VECTORS

VECTOR	DESCRIPTION	CLASS
Direct injection	User instructs the assistant to ignore its rules, reveal its prompt or act without restriction	Input
Indirect injection	Instructions hidden in content the assistant reads rather than in what the user typed	Content
Multi-turn injection	Payload assembled gradually across many messages so no single turn appears malicious	Input
Multilingual injection	Restricted instructions expressed in another language or script to evade keyword matching	Input
Encoded injection	Payload wrapped in hex, ROT, URL-encoding, homoglyphs or nested encodings	Input
Base64 injection	Instructions supplied encoded with a request to decode and then follow them	Input
Unicode obfuscation	Confusable characters, bidirectional overrides and normalisation tricks disguising intent	Input
Invisible-character attacks	Zero-width, tag and control characters hiding instructions from human reviewers	Content
Prompt delimiter attacks	Fake block markers, closing tags or separators intended to break out of the data region	Input
Context poisoning	Untrusted text placed into context so later turns treat it as established fact or policy	Context
Instruction hierarchy attacks	Claims that a later, longer or “updated” instruction outranks the system policy	Context
Role impersonation	Attacker asserts a privileged role such as staff, auditor, developer or account owner	Input
Fake system messages	Text formatted to imitate a system turn or platform notice	Input
Fake developer messages	Pseudo-debug, test-mode or maintenance instructions requesting unrestricted behaviour	Input
Fake administrator instructions	Forged approvals, override codes or “authorised by security” claims	Input
Urgency-based manipulation	Fabricated emergencies, deadlines or threats pressuring the assistant to skip a control	Social
Social engineering	Sympathy, authority, reciprocity or persistence aimed at policy erosion over a conversation	Social
Tool-result injection	Malicious instructions embedded in data returned by a tool or upstream API	Content
RAG poisoning	Knowledge-base or indexed document seeded with instructions to alter behaviour on retrieval	Content
Uploaded-document injection	Instructions inside PDFs, images, spreadsheets, metadata or OCR-extracted text	Content
Webpage injection	Fetches page containing hidden text, comments or markup targeting the summariser	Content

Every vector above has at least one corresponding test case in §20. New vectors discovered in testing or in the wild are added to both this table and the test catalogue.

04 Injection Defense & Instruction Hierarchy

SECTION 04 CONTINUED

FIXED INSTRUCTION HIERARCHY

- 1 **Platform security policy** — immutable at runtime; cannot be disabled, overridden, revealed in full or “updated” by any conversational input
- 2 **Backend-enforced business rules** — authorization, pricing, eligibility and state transitions decided by trusted services
- 3 **Application system prompt** — scope, tone and permitted task boundaries for the SaveHatke assistant
- 4 **Authenticated user request** — treated as a request, never as an instruction that can widen the assistant's own permissions
- 5 **All retrieved and returned content** — documents, pages, files, tool results, memory: inert data at the lowest priority, always

INPUT HANDLING & SANITISATION

CONTROL	REQUIREMENT	LEVEL
Unicode normalisation	Normalise form, strip zero-width and control characters, reject or neutralise bidirectional overrides, flag homoglyph clusters	MUST
Structural separation	Untrusted content is passed in clearly delimited, non-instructional regions with provenance labels; delimiters are generated server-side and stripped from user input	MUST
Decoded-content screening	Content the assistant is asked to decode is screened after decoding and still treated as data, never as instructions to execute	MUST
Length & volume limits	Message, document and context size caps reduce burial attacks and cost exposure (§17)	MUST
Injection heuristics	Layered detection across languages and encodings for override phrasing, forged role markers, policy-extraction attempts and hidden-instruction patterns	SHOULD
Multi-turn correlation	Detection considers the conversation, not only the latest message, so gradual assembly is scored as one attempt	SHOULD
System-prompt confidentiality	Internal policy text, tool schemas and infrastructure detail are not disclosed; refusals stay brief and do not enumerate what was blocked	MUST
Refusal consistency	Repetition, rephrasing, hypotheticals, role-play framing and claimed authority do not change a refusal outcome	MUST

DEFENSE IN DEPTH BEHIND THE MODEL

Injection Is Contained, Not Merely Detected

Detection reduces attempts; it is never the last line. Even a fully successful injection cannot read another user's record, because retrieval and tools are authorization-scoped (§5, §7, §8). It cannot change a price, balance or order state, because the backend recalculates and re-authorizes independently (§10, §13). It cannot reach infrastructure, because the runtime is isolated with default-deny egress (§3, §15). It cannot hide, because every attempt and every tool call is logged (§18).

ASSISTANT BEHAVIOURAL PROHIBITIONS

- × Will not reveal or paraphrase internal security policy
- × Will not accept claimed staff, admin or developer status
- × Will not honour “override”, “test mode” or “debug” requests
- × Will not follow instructions found inside documents or pages
- × Will not confirm, restate or store credentials or OTPs
- × Will not describe internal endpoints, schemas or tool names
- × Will not speculate about other users' accounts or data
- × Will not promise an outcome the backend has not confirmed
- × Will not treat urgency or pressure as authorisation
- × Will not re-attempt a blocked operation by another route

05 Tools, Agents & Excessive-Agency Protection

MANDATORY PIPELINE

A tool call is a privileged request that happens to have been proposed by a model. It is therefore subjected to the full server-side gauntlet below. Every stage is enforced in trusted backend code, in order, on every call — including retries, follow-up calls and calls made inside a multi-step plan.

EVERY AI TOOL CALL MUST PASS

- 1 Identity verification** — a valid, unrevoked server-side session resolved to a concrete account; anonymous or expired context reaches no private tool
- 2 Authorization check** — the acting identity holds the specific permission the tool declares, evaluated deny-by-default
- 3 Resource ownership check** — every referenced object is confirmed to belong to, or be explicitly shared with, the acting identity
- 4 Operation allowlist** — the requested tool and action exist in the static server-side registry for this surface and this role
- 5 Argument schema validation** — strict types, ranges, formats, enum membership and size caps; unknown fields rejected rather than ignored
- 6 Business-rule validation** — state machine legality, eligibility, timing windows, quantity and value limits, and recalculation of every derived amount
- 7 Rate-limit check** — per-user, per-session, per-tool, per-endpoint and global budgets, plus concurrency ceilings (§16, §17)
- 8 Risk evaluation** — sensitivity, reversibility, monetary value and anomaly signals decide whether to proceed, require step-up, or queue for human review
- 9 Tool execution** — performed by the backend as the end user, within timeout, with no privilege the user does not independently hold
- 10 Output validation** — result schema-checked, field-allowlisted, redacted for the acting identity, and marked untrusted before re-entering context (§7, §9)
- 11 Audit logging** — actor, session, tool, arguments summary, decision, outcome, correlation ID and timing written to a tamper-resistant log (§18)

Pipeline Rule

Any stage that cannot complete is a denial. A stage that errors, times out, cannot reach its dependency or returns an indeterminate result fails the call closed, returns a generic message to the user and raises a security event. Stages are never skipped for performance, never cached across identities, and never bypassed because an earlier turn in the same conversation succeeded.

PIPELINE INVARIANTS

INVARIANT	WHY IT MATTERS
Model influence stops at stage 0	The model can only propose a name and arguments; it cannot alter the registry, the policy or any gate
No ambient authority	Tools run as the user, so a compromised prompt inherits exactly the user's own rights and nothing more
Independent recalculation	Prices, totals, balances, discounts and eligibility are computed server-side, so stated values in chat are irrelevant to the outcome
Deterministic auditability	Every attempt, allowed or denied, is reconstructible from logs with a correlation ID linking chat turn to backend effect

05

Capability Prohibitions & Execution Limits

SECTION 05 CONTINUED

CAPABILITIES THE AI NEVER HOLDS

- × No arbitrary tool names — unregistered names are refused
 - × No arbitrary function names or dynamic dispatch
 - × No dynamic privilege creation of any kind
 - × No AI-controlled role changes
 - × No AI-controlled permission changes
 - × No AI-controlled payout approval
 - × No AI-controlled refund approval
- × No AI-controlled ownership transfer
 - × No AI-controlled account deletion or merge
 - × No AI-initiated credential or MFA changes
 - × No unrestricted batch or bulk operations
 - × No unbounded enumeration or export of records
 - × No direct writes bypassing business logic
 - × No self-modification of tools, schemas or policy

EXECUTION LIMITS

LIMIT	REQUIREMENT	LEVEL
Maximum tool calls per request	Hard ceiling per user message and per conversation; exceeding it ends the turn safely and raises a security event	MUST
Maximum tool execution time	Per-tool wall-clock budget enforced server-side, independent of the model's own latency	MUST
Tool timeout	Timeouts terminate the call, release resources and return a deterministic failure rather than a partial result	MUST
Tool circuit breaker	Repeated failures, timeouts or denials on a tool trip a breaker that disables it for a cool-down window and alerts on-call	MUST
Loop & recursion guard	Repeated identical or cyclic call patterns are detected and stopped; agentic plans have a bounded depth and step count	MUST
Batch size caps	Any tool that accepts a collection enforces a maximum item count, with pagination rather than unbounded reads	MUST
Result size caps	Tool results are truncated to a documented maximum before entering context, protecting both cost and exfiltration surface	MUST
Concurrency ceiling	Parallel tool execution per user and globally is bounded to protect shared capacity (\$17)	MUST
Idempotency for state change	Every mutating tool requires an idempotency key so a retried or duplicated call cannot double-apply an effect (\$13)	MUST

HUMAN AUTHORITY FOR HIGH-RISK OPERATIONS

OPERATION	REQUIRED PATH
Payouts & withdrawals	Backend approval workflow with step-up authentication, velocity checks and manual review above threshold (\$13, \$22)
Refunds & credits	Policy engine plus authorised staff action; the assistant may only explain status and eligibility
Role, permission & ownership changes	Admin surface only, outside the chatbot, with audit and two-person approval where applicable
Bulk data export	Authenticated, rate-limited, logged export path with re-authentication; never assembled turn-by-turn in chat

Agency Rule

AI output can never authorize an operation. A tool executes because the pipeline in this section permitted it, not because the model asked. If a capability would be dangerous when driven by a fully attacker-controlled model, that capability does not belong behind the assistant at all.

06 AI Memory & Conversation Context Security

NEW IN V4.0

Anything persisted becomes part of tomorrow's input. Memory therefore extends an attacker's window from one message to every future message, and is treated as an untrusted store that is authorization-scoped on write and on read.

MEMORY TRUST RULES

RULE	REQUIREMENT	LEVEL
History is data, not instruction	Previous conversation content is never treated as a trusted instruction, policy statement or grant of permission	MUST
Memory cannot override policy	No stored preference, summary or "remembered rule" can weaken, disable or reinterpret a security control	MUST
Separation of stores	User memory is stored and loaded separately from security policy and system instructions, with different write paths and different trust levels	MUST
Authorization before retrieval	Private memory is fetched only after the acting identity is confirmed to own the conversation being loaded	MUST
Re-check on historical access	Authorization is re-evaluated when historical context is retrieved — access granted in an earlier turn does not persist if entitlements have since changed	MUST
Cross-user isolation	One user's memory can never surface in another user's session; conversation keys are namespaced per account and never guessable	MUST
Long history is untrusted input	Extended transcripts are scanned and normalised on reload with the same input controls applied to fresh messages (§4)	SHOULD
Summarisation safety	Automatic summaries carry no instructions forward; they record facts and outcomes, not directives, and are regenerated rather than appended blindly	SHOULD

NEVER WRITTEN TO MEMORY OR TRANSCRIPTS

- × Passwords or password hints
- × One-time passcodes and verification codes
- × Recovery codes and MFA seeds
- × API keys, tokens and session identifiers
- × Full card numbers, CVV or expiry data
- × Bank, UPI or payout destination secrets
- × Government identifier numbers
- × Any secret the platform itself holds (§12)

DATA MINIMISATION, RETENTION & DELETION

CONTROL	REQUIREMENT
Minimisation	Store the least context needed for continuity; sensitive values are referenced by opaque identifier rather than copied into the transcript
Retention limits	Documented maximum retention for transcripts, derived summaries, embeddings and logs, with automated expiry rather than manual cleanup
User deletion	Users can delete eligible conversation data; deletion propagates to summaries, embeddings and caches, with legal and financial records retained per policy and clearly scoped
Encryption & access	Conversation stores encrypted at rest, access restricted to the services that need them, and staff access audited (§11, §22)

Memory Poisoning

An attacker seeds durable content — a "preference", a fabricated prior approval, a claimed staff confirmation, a hidden instruction inside an uploaded file — hoping it is reloaded later and treated as trusted. Defenses: write-time screening of anything persisted, provenance labels retained with stored content, no instruction-shaped content in summaries, authorization re-checked on every reload, and the absolute rule that memory can never satisfy stages 1–8 of the tool pipeline in §5. Suspected poisoning is a detection signal (§18) and has a response playbook (§19).

07

AI Data Exfiltration Prevention

NEW IN V4.0

A conversational interface is an attractive extraction channel: it accepts natural language, returns free-form text and invites persistence. Prevention is layered — restrict what can be retrieved, then restrict what can be emitted.

PROTECTED ASSET CLASSES

- Other users' accounts and profiles
 - Private orders and order history
 - Wallet balances and transaction records
 - Payout details and destinations
 - Support records and ticket history
 - Internal documents and runbooks
- Restricted knowledge-base content
 - Staff identities, rosters and notes
 - System and application configuration
 - API credentials and provider keys
 - Database names, schemas and queries
 - Infrastructure, hostnames and topology

PREVENTION CONTROLS

CONTROL	REQUIREMENT	LEVEL
Authorization-aware retrieval	Retrieval and tool reads are filtered by the acting identity's entitlements before data ever reaches model context — not filtered afterwards in the answer	MUST
Cross-user isolation	Queries, caches, embeddings, session state and rate-limit keys are namespaced per account so foreign records are unreachable by construction	MUST
Tenant isolation	Where partner or multi-tenant data exists, tenant scope is bound to the session and enforced at query level (§2, §11)	MUST
PII minimisation	Only the fields required for the answer are retrieved; identifiers are masked to the minimum useful form, such as last-four or partial contact display	MUST
Response filtering	Outbound text is inspected against the acting identity's entitlements before delivery; anything unauthorised is removed, not merely reworded	MUST
Sensitive-data detection	Pattern and classifier checks for card numbers, tokens, key material, credentials and bulk personal data in outbound content	SHOULD
Secret scanning on output	Known secret formats are blocked in responses and in logs; a match is a high-severity security event, not a formatting problem (§12, §18)	MUST
Output redaction	Redaction applied consistently to chat responses, tool summaries, error text, citations and any file the assistant produces	MUST
Volume & velocity controls	Per-turn and per-session caps on records returned, plus anomaly detection on extraction-shaped behaviour (§16, §18)	MUST

WATCHED EXFILTRATION CHANNELS

CHANNEL	COUNTERMEASURE
Direct request & role-play framing	Authorization-scoped retrieval makes the data unavailable regardless of framing; refusals stay consistent (§4)
Aggregation across many turns	Session-level volume accounting, not per-message accounting; velocity anomalies escalate
Error and debug text	Generic user-facing errors; internal detail confined to server logs with secrets redacted (§10)
Outbound links, images & markup	URL validation and sanitisation prevent data being smuggled in a rendered link or remote asset (§9, §15)
Generated files & exports	Produced artefacts pass the same authorization and redaction checks as chat text

08 RAG & Knowledge Base Security

RETRIEVAL LAYER

Retrieval decides what the model gets to see. It is therefore an access-control boundary first and a relevance problem second. Permissions are evaluated before a search runs, not applied to its results.

RETRIEVAL CONTROLS

CONTROL	REQUIREMENT	LEVEL
Authorization before retrieval	The acting identity's entitlements constrain the candidate set before search executes; unauthorised documents are never scored, ranked or embedded into context	MUST
Document-level permissions	Every indexed document carries an explicit audience: public, authenticated, staff, or restricted with a named grant	MUST
Chunk-level permissions	Where a document mixes audiences, permissions are carried on the chunk and inherited by its embedding	SHOULD
Tenant isolation	Indexes and vector namespaces are partitioned per tenant; a query cannot traverse partitions even with a crafted filter	MUST
Restricted-document separation	Internal runbooks, staff procedures and security material live in indexes the customer-facing assistant cannot query at all	MUST
Source validation	Only approved sources are ingested; ad-hoc ingestion from arbitrary URLs or user uploads into the shared index is prohibited	MUST
Document provenance	Origin, uploader, approval state, ingestion time and checksum are recorded and retained alongside every chunk	MUST
Version control	Knowledge content is versioned with diff review and rollback; the active version in production is always identifiable	MUST
Security review of uploads	New or changed knowledge is reviewed before publication, with a higher bar for anything describing policy, limits, pricing or entitlements	MUST
Prompt-injection scanning	Ingested content is scanned for instruction-shaped text, hidden characters, encoded payloads and role-impersonation markers (§4)	MUST
Poisoned-document detection	Anomalous edits, sudden policy-like additions, unexpected authority claims and unusual retrieval spikes are flagged for review	SHOULD
Retrieval-result validation	Results are size-capped, provenance-labelled and marked untrusted before entering context; content that failed scanning is dropped	MUST
Retrieval logging	Query, identity, index, permission decision, documents returned and correlation ID are logged for investigation (§18)	MUST
Citation integrity	Cited sources must be ones actually retrieved and permitted for that identity; links are validated before rendering (§9, §15)	SHOULD
No policy change via content	Retrieved documents cannot alter security policy, instruction hierarchy or tool availability under any circumstances	MUST
No privilege escalation via content	Retrieved documents cannot grant roles, unlock capabilities, approve operations or satisfy any stage of the §5 tool pipeline	MUST

Retrieval Rule

A document describing a permission is not a permission. Knowledge content informs answers; it never confers authority. If retrieved text and platform policy disagree, platform policy wins and the discrepancy is raised for content review.

09

AI Output Security

NEW IN V4.0

Governing Rule

AI output is untrusted data. It is rendered as content, parsed against a schema, or discarded. It is never executed, never trusted as an authorization decision, and never treated as a financial or state-changing instruction.

NEVER DONE WITH MODEL OUTPUT

- ×

Never executed as code in any language
- ×

Never executed as SQL or any query language
- ×

Never executed as a shell or system command
- ×

Never used as an authorization decision
- ×

Never used as a financial decision or approval
- ×

Never used to construct dynamic privileges
- ×

Never written directly to a datastore unvalidated
- ×

Never rendered as raw HTML into the page
- ×

Never used to build file paths or system calls
- ×

Never used to select which tenant or account is read

OUTPUT VALIDATION & RENDERING CONTROLS

CONTROL	REQUIREMENT	LEVEL
Structured-output validation	Any machine-consumed output is validated against a strict schema: expected types, enums, ranges and required fields, with unknown fields rejected	MUST
Server-side re-validation	Values extracted from output are re-validated and re-authorized in backend logic before any effect, exactly as if a user had supplied them	MUST
Contextual escaping	All rendered content is escaped for its destination context — HTML text, attribute, URL, script-free templates — by default rather than by exception	MUST
HTML sanitisation	If any markup is permitted, it passes a strict allowlist sanitiser: no script, no event handlers, no style injection, no data URIs, no inline SVG	MUST
XSS prevention	Framework auto-escaping left enabled, raw-HTML APIs prohibited in chat rendering, and a restrictive Content Security Policy enforced (§10)	MUST
Markdown containment	Markdown is rendered by a sanitising renderer with images, links and embeds restricted; unknown constructs are shown as plain text	MUST
URL & link validation	Every generated or cited URL is validated for scheme, host and format before rendering; unresolvable or disallowed targets are shown as inert text (§15)	MUST
No unsafe embeds	Iframes, objects, embeds, remote fonts and remote scripts are never created from model output; third-party media is not auto-loaded	MUST
Redaction before delivery	Output passes exfiltration and secret checks before it reaches the user or any log sink (§7, §12)	MUST
Claim discipline	The assistant does not assert that an operation succeeded unless a backend confirmation was returned; it reports pending or failed states honestly	MUST
Safe failure text	User-facing errors are generic and actionable; stack traces, identifiers, queries and internal hostnames are never surfaced (§10)	MUST

Streaming Output

Streamed responses are sanitised incrementally with the same rules as complete responses. Partial content is never rendered through a raw-HTML path for speed, buffers are size-capped, and a response cancelled mid-stream is discarded rather than partially committed.

10 Web, API & Infrastructure Hardening

PLATFORM LAYER

The chat endpoint is an API endpoint and receives no special treatment. Every control that applies to the platform's other authenticated APIs applies to it, plus the AI-specific limits in §17.

API SECURITY CONTROLS

CONTROL	REQUIREMENT	LEVEL
Authentication	Every non-public route requires a validated server-side session or service credential; no route relies on obscurity or a client flag	MUST
Function-level authorization	Each route declares the permission it needs and enforces it before any logic runs (§2)	MUST
Object-level authorization	Every identifier in a path, query or body is ownership-checked against the acting identity (§2)	MUST
Strict request schemas	Typed, versioned schemas with allowlisted fields; unknown or extra fields are rejected rather than silently ignored	MUST
Strict response schemas	Responses are serialised from explicit field allowlists so internal columns cannot leak through model or ORM changes	MUST
Server-side validation	All validation is authoritative on the server; client-side validation is treated purely as a usability aid	MUST
API versioning	Breaking changes ship as new versions with a documented deprecation path; old versions retain their own security controls until retired	SHOULD
Request & response size limits	Body, header, field, upload and response sizes are capped at the edge and at the application layer	MUST
Timeout & concurrency limits	Per-request timeouts and per-identity concurrency ceilings prevent slow-request resource exhaustion	MUST
Rate limits	Layered per-IP, per-account, per-session and per-endpoint limits with a global backstop (§16)	MUST
Replay protection	Where required, requests carry nonces or timestamps with bounded skew; webhooks additionally verify signatures (§13)	MUST
Idempotency	Sensitive and mutating operations accept an idempotency key and return the original result on replay instead of re-applying it	MUST
Error-message minimisation	Generic client errors, correlation IDs for support, full detail only in server logs with secrets redacted	MUST
Data-in-transit hygiene	No sensitive information in URLs, query strings, referrers or redirect targets; no secrets in client-side code; no credentials in logs	MUST

EDGE & INFRASTRUCTURE HARDENING

- HTTPS everywhere with HSTS and modern TLS only
- WAF and bot protection in front of the chat API
- Restrictive Content Security Policy
- Clickjacking, MIME-sniffing and referrer controls
- Strict CORS allowlist; no wildcard with credentials
- CSRF protection on all state-changing browser routes
- Default-deny egress from application and AI runtimes
- Network segmentation between tiers
- Admin and internal surfaces not publicly routable (§22)
- Debug endpoints, docs and profilers disabled in production
- Patch and base-image update cadence (§21)
- Infrastructure changes via reviewed code, not console edits

11 Database Security

NEW IN V4.0

The datastore is a protected resource reached only through backend services. Access is least-privilege, parameterised, ownership-checked, encrypted and monitored.

ACCESS & PRIVILEGE

CONTROL	REQUIREMENT	LEVEL
No direct AI access	The AI runtime holds no database credentials, no connection string and no query capability; it reaches data only through allowlisted tools (§3, §5)	MUST
Separate application roles	Distinct database roles per service and per purpose, so a single compromised component cannot reach unrelated data	MUST
Least privilege	Each role holds only the object and statement privileges it needs; no application role holds schema-modification or superuser rights	MUST
Read/write separation	Read-only credentials for reporting, analytics and read paths; write credentials confined to the services that legitimately mutate state	SHOULD
Parameterised queries	All queries use parameter binding; string-concatenated SQL is prohibited, including in migrations, jobs and admin tooling	MUST
ORM protections retained	ORM escaping and query building are not bypassed; raw-query escape hatches require review and parameter binding	MUST
NoSQL injection defense	Operator injection is blocked by validating and coercing types before query construction; user input never becomes a query operator	MUST
Object ownership checks	Every read and write is constrained by owner or grant at the query layer and re-checked in business logic (§2)	MUST
Row-level authorization	Where the datastore supports it, row-level policies provide a second enforcement layer independent of application code	SHOULD
Production isolation	Production data and credentials are fully separated from development, staging and test; production data is not copied into lower environments unmasked	MUST
No credentials in clients or prompts	Database credentials never appear in frontend code, mobile bundles, chatbot prompts, tool descriptions, error text or logs (§12)	MUST
Secret rotation	Database credentials are rotated on schedule and immediately on suspicion, with rotation exercised rather than assumed (§12)	MUST
Activity monitoring	Query error spikes, privilege changes, bulk reads, off-hours access and unusual client fingerprints are monitored and alerted (§18)	MUST

ENCRYPTION, BACKUP & RECOVERY

- Encryption at rest for databases and volumes
 - Encryption in transit for every database connection
 - Field-level protection for the most sensitive attributes
 - Encrypted backups with keys held separately
 - Backup access restricted to a named, audited group
- Backups isolated from production credentials
 - Restoration tested on a documented schedule
 - Retention and expiry policy enforced automatically
 - Recovery objectives defined and validated
 - Restore drills recorded in the release gate (§24)

Database Rule

No component reaches the database with more authority than its job requires, and the AI reaches it with none. Every data path is a reviewed, parameterised, ownership-checked backend operation — never a query the assistant composed.

12

Secrets & Key Management

NEW IN V4.0

Secrets are the shortest path from a small mistake to a large incident. They live in a server-side secret manager, are delivered only to the workloads that need them, and are never placed anywhere the AI or the browser can read.

SECRETS UNDER MANAGEMENT

- AI provider API keys

▸ Database credentials

▸ Payment provider keys

▸ Webhook signing secrets

▸ Token and request signing keys

▸ Encryption keys and key-encryption keys
- OAuth client secrets

▸ Internal service-to-service credentials

▸ Session-signing secrets

▸ Object storage and CDN credentials

▸ Notification and messaging provider keys

▸ Monitoring and log-shipping credentials

HANDLING RULES

RULE	REQUIREMENT	LEVEL
Secret manager only	Secrets are stored in a managed secret store with access policy and audit; not in source, not in images, not in plain configuration files	MUST
Never exposed to the browser	No secret is shipped in frontend bundles, HTML, environment variables readable by the client, or public configuration endpoints	MUST
Never exposed to the AI	Secrets are not present in prompts, system instructions, tool descriptions, tool results, retrieved documents or conversation memory	MUST
Provider keys held by a gateway	Calls to the AI provider are made by a dedicated server-side gateway that holds the key; the runtime never sees it (§3)	MUST
Never committed	Pre-commit and CI secret scanning block commits; history is scanned periodically and any historical exposure is treated as a live secret	MUST
Automatic scanning	Repositories, build artefacts, container images, logs and outbound responses are scanned continuously for secret patterns (§7, §21)	MUST
Environment separation	Production and non-production credentials are entirely distinct; a development key can never reach a production resource	MUST
Least-privilege scope	Each secret is scoped to the narrowest capability and shortest lifetime that works, with per-workload identities rather than shared credentials	MUST
No secrets in logs or errors	Structured logging redacts known secret fields by default, and outbound error text never contains credential material (§10, §18)	MUST

ROTATION & EMERGENCY REVOCATION

PROCEDURE	REQUIREMENT
Scheduled rotation	Documented rotation interval per secret class, automated where the provider supports it, with overlapping validity to avoid outage-driven shortcuts
Emergency revocation	A rehearsed procedure to revoke and re-issue any secret quickly, with a named owner per secret class and a tested runbook (§19)
Exposure response	Any suspected exposure triggers immediate revocation, forced session invalidation where relevant, and an investigation of what the credential could reach
Verification	Rotation and revocation are exercised at least once per cycle in a controlled test, and evidence is recorded in the release gate (§24)

13 Payment & Financial Security

HIGH-RISK SURFACE

Critical Rule

AI output can never authorize a financial transaction. The assistant may explain fees, describe status, summarise an invoice and guide a user to the correct flow. Money moves only when trusted backend services have independently authenticated the user, authorized the operation, recalculated the amount and verified the payment state with the provider.

NEVER REACHES THE ASSISTANT

- × Raw card numbers or PANs
- × CVV or card security codes
- × Card expiry with cardholder data
- × One-time passcodes for payment authentication
- × Net banking or UPI credentials and PINs
- × Full bank account or payout secrets

PAYMENT CONTROLS

CONTROL	REQUIREMENT	LEVEL
Hosted provider flow	Card and bank data are captured only in the payment provider's hosted flow or SDK; SaveHatke systems and the assistant never receive or store raw instrument data	MUST
Server-side price calculation	Amounts, taxes, shipping, discounts and totals are computed server-side from authoritative records; any value stated in chat or sent by a client is ignored	MUST
Server-side payment verification	Order state advances only after the backend confirms payment with the provider through a trusted channel; client-reported success is never sufficient	MUST
Webhook signature verification	Every webhook is signature-verified, timestamp-checked, replay-protected and matched to a known intent before it changes any state	MUST
Idempotency protection	Payment, refund, wallet and payout operations are idempotent on a stable key so retries and duplicate webhooks apply exactly once	MUST
Duplicate-payment protection	Duplicate charges are detected and reconciled automatically, with alerting when reconciliation cannot complete	MUST
Transaction-state validation	A strict state machine governs legal transitions; out-of-order, repeated or impossible transitions are rejected and alerted	MUST
Server-side ownership validation	Every order, invoice, wallet entry and payout is confirmed to belong to the acting identity before it can be viewed or acted upon (§2)	MUST
Payout approval workflow	Payouts follow a defined workflow with step-up authentication, destination cool-down, threshold-based manual review and separation of requester from approver (§22)	MUST
Refund approval workflow	Refunds require policy-engine eligibility plus authorised staff action; the assistant can only report eligibility and status	MUST
Velocity checks	Rate and value ceilings per user, instrument, device and destination over rolling windows, with automatic holds when exceeded	MUST
Suspicious transaction detection	Anomaly scoring on amount, timing, geography, device, new destinations and refund or chargeback patterns feeds review queues (§18)	SHOULD
Manual review for high risk	High-value or high-risk financial events are queued for human decision; the queue fails closed if review capacity is unavailable	MUST
Financial audit trail	Append-only records of every financial event with actor, source, amount, provider reference, decision path and correlation ID, retained per policy (§18)	MUST

14 File & Image Security

HIGH-RISK SURFACE

Uploads combine three risks at once: hostile parsing input, hostile stored content, and hostile instructions aimed at the assistant that reads them. All three are handled explicitly.

UPLOAD & PROCESSING CONTROLS

CONTROL	REQUIREMENT	LEVEL
File-type allowlist	Only explicitly permitted types are accepted; everything else is rejected rather than best-effort handled	MUST
MIME validation	Declared content type is checked, and disagreement with the actual content is a rejection	MUST
Magic-byte validation	File signature is inspected server-side; extension and client-declared type are never trusted	MUST
File-size limits	Per-file and per-request size caps enforced at the edge and in the application (§17)	MUST
Image dimension limits	Maximum pixel dimensions and total pixel count enforced before decoding to prevent memory exhaustion	MUST
Filename normalisation	Path separators, traversal sequences, control characters, unicode tricks and overlong names are stripped or rejected	MUST
Random server-side filenames	Stored objects use server-generated opaque identifiers; the original name is retained only as display metadata	MUST
Malware scanning	Uploads are scanned before processing or availability; unscannable content is quarantined, not passed through	MUST
Sandboxed processing	Parsing, conversion and thumbnailing run in a confined worker with no credentials, no egress and strict CPU, memory and time budgets	MUST
Isolated OCR & vision	Text and vision extraction run in the same confined tier; extracted text is treated as untrusted input and injection-scanned (§4)	MUST
Never execute uploads	Uploaded content is never executed, interpreted, included, imported or rendered as active content in any context	MUST
Archive-bomb protection	Decompression is bounded by output size, entry count, nesting depth and expansion ratio, with hard abort on breach	MUST
Content validation	Structure is validated against the declared type; malformed documents are rejected with a generic error	MUST
Metadata & privacy handling	EXIF and embedded metadata are stripped from shared or public images; retained only where a documented purpose requires it	SHOULD
Safe storage	Objects are stored outside executable web roots in dedicated storage, served through authorized, expiring URLs with a non-executing content type	MUST
Access control on retrieval	Every download is ownership-checked; object identifiers are unguessable and never a substitute for authorization (§2)	MUST

Documents Are Data, Never Instructions

Text recovered from a PDF, image, spreadsheet or document's metadata is content the assistant may summarise. It cannot direct the assistant, unlock a tool, assert an approval or claim authority — the instruction hierarchy in §4 places all extracted content at the lowest priority, and the tool pipeline in §5 will not accept it as justification for any operation.

15

Web Fetch, URL & SSRF Security

HIGH-RISK SURFACE

Any feature that fetches a user-supplied URL turns the assistant into a request proxy inside the network perimeter. Fetching is therefore explicitly allowlisted, validated at resolution time and bounded in size and duration.

BLOCKED DESTINATIONS

- ×

Private and reserved IP ranges
- ×

Internal service hostnames and service discovery names
- ×

Loopback addresses and localhost aliases
- ×

Non-HTTPS schemes, including file, gopher, ftp and data
- ×

Link-local addresses, including IPv6 equivalents
- ×

Ports outside the permitted set
- ×

Cloud instance metadata endpoints
- ×

Hosts resolving into any blocked range after DNS lookup

FETCH CONTROLS

CONTROL	REQUIREMENT	LEVEL
HTTPS only	Only HTTPS URLs are fetched, with certificate validation enforced and no downgrade path	MUST
Hostname validation	URLs are parsed with a strict parser; malformed, obfuscated, credential-embedding and encoded-host forms are rejected	MUST
DNS resolution validation	Resolved addresses are checked against the blocklist immediately before connecting, and the validated address is the one connected to — closing the rebinding window	MUST
Redirect validation	Every hop is re-validated against the same rules; redirects to blocked destinations abort the fetch	MUST
Redirect limit	A low maximum hop count, with abort and log on breach	MUST
Egress filtering	The fetching workload runs behind default-deny egress so an application-layer bypass still cannot reach internal networks (§3)	MUST
Fetch timeout	Connection and total-transfer timeouts are enforced independently of the model's latency budget	MUST
Response-size limit	Transfers are truncated at a hard byte ceiling; oversized responses are discarded rather than partially parsed	MUST
Content-type restriction	Only expected document types are processed; unexpected types are dropped without parsing	MUST
No credential forwarding	Outbound fetches carry no session cookies, tokens, internal headers or platform credentials	MUST
Fetched content is untrusted	Retrieved pages are injection-scanned, provenance-labelled and treated as lowest-priority data (§4, §8)	MUST
Outbound request logging	Requesting identity, requested URL, resolved address, decision, status, byte count and correlation ID are logged for every attempt (§18)	MUST
Rate & concurrency limits	Per-user and global fetch budgets prevent the platform being used as an amplifier or scanner (§16)	MUST

SSRF Rule

Validate at connect time, not at parse time. A URL that passed inspection is not the same thing as a connection that is safe: the destination is re-checked after DNS resolution, on every redirect, and every fetch is bounded, credential-free and logged.

16 Bot Abuse & Automation Defense

LAYERED LIMITS

Limits are layered so that evading one dimension does not grant unlimited access. An attacker rotating addresses still meets account limits; an attacker rotating accounts still meets global and cost limits.

LIMIT LAYERS

LAYER	APPLIES TO	LEVEL
IP limits	Request rate and burst per address and per network block, applied at the edge before application cost is incurred	MUST
Account limits	Messages, tool calls, uploads, fetches and financial attempts per account over rolling windows	MUST
Session limits	Per-session ceilings and abnormal-session detection, so a single stolen session cannot be worked indefinitely	MUST
Device signals	Device and client fingerprint signals used as risk input for throttling and challenge decisions, never as an identity claim	SHOULD
Endpoint-specific limits	Tighter budgets on expensive or sensitive routes: login, recovery, payment, payout, upload, fetch, export	MUST
Tool-specific limits	Each tool carries its own budget so one costly tool cannot consume the whole allowance (\$5)	MUST
Global limits	Platform-wide ceilings act as a backstop when a distributed attack stays under every per-identity limit	MUST
Concurrency limits	Simultaneous conversations, tool executions, uploads and fetches bounded per identity and globally	MUST
AI token limits	Input, output and total context budgets per message, per conversation and per account per period (\$17)	MUST
File-processing limits	Uploads per request and per window, plus total bytes and processing seconds per account (\$14)	MUST
Cost ceilings	Monetary ceilings per user and platform-wide, enforced independently of request counts (\$17)	MUST
Automation detection	Timing regularity, identical payload reuse, impossible interaction speed, headless signals and enumeration patterns raise a risk score	SHOULD

PROGRESSIVE RESPONSE LADDER

- 1 Soft throttle** — slow the identity, keep the experience functional for legitimate use
- 2 Exponential backoff** — increasing delay per violation within the window, reset only after sustained good behaviour
- 3 Temporary challenge** — interactive verification on the suspicious surface, never as a substitute for authorization
- 4 Feature restriction** — disable the abused capability while leaving the account otherwise usable
- 5 Temporary block** — time-boxed denial of the identity or address, logged with reason and expiry
- 6 Emergency shutdown** — documented kill switch to disable a tool, the AI surface or the chatbot entirely, tested before release (\$17, \$24)

Limit breaches return generic messages. Responses never disclose which threshold was hit, what the limit is, or when it resets in a way that supports precise tuning by an attacker.

17 AI Denial-of-Service & Cost Protection

NEW IN V4.0

For an AI feature, availability and cost are the same control problem: every request consumes metered capacity. Unbounded consumption is treated as a security issue, not only a billing surprise.

CONSUMPTION CEILINGS

CEILING	REQUIREMENT	LEVEL
Maximum input tokens	Per-message input cap enforced before the provider call; oversized input is truncated or refused deterministically	MUST
Maximum output tokens	Hard output cap per response, preventing unbounded generation loops	MUST
Maximum conversation context	Total context bounded by trimming or summarising history under the memory rules in §6	MUST
Maximum file size	Per-file byte ceiling checked before any processing begins (§14)	MUST
Maximum files per request	Count ceiling per message and per conversation, with total-bytes accounting	MUST
Maximum tool calls	Per-turn and per-conversation call budget from the §5 pipeline	MUST
Maximum concurrent AI jobs	Per-user and global concurrency ceilings with a bounded queue that rejects rather than growing without limit	MUST
Per-user cost ceiling	Rolling monetary or token budget per account; on breach the account is throttled to a reduced tier and the event is logged	MUST
Global cost ceiling	Platform-wide spend ceiling per window that degrades the feature gracefully rather than exhausting the budget	MUST
Budget alerts	Threshold alerts at defined percentages of daily and monthly budget, routed to an owner who can act (§18)	MUST
Spend anomaly detection	Detection on sudden growth in tokens per user, tool calls per session, retries, or cost per conversation against a rolling baseline	SHOULD
Timeout controls	Provider-call, tool-call and total-turn timeouts with deterministic user-facing failure and no silent retry storms	MUST
Retry discipline	Bounded retries with jittered backoff; retries counted against the same budgets as first attempts	MUST
Circuit breakers	Provider errors, latency breaches or cost-rate breaches trip a breaker that sheds load and serves a degraded, non-AI path	MUST
Emergency AI shutdown	A tested kill switch disables the AI surface or an individual tool without deploying code, with a documented owner and re-enable procedure	MUST

Graceful Degradation Order

Under pressure the platform sheds capability in a defined order: reduce optional enrichment, then disable expensive tools, then shorten context and output, then queue with clear user messaging, then serve non-AI support paths. Degradation never removes a security control — authentication, authorization, validation and logging stay fully enforced in every degraded mode, and if they cannot be enforced the request is refused (§1).

18 Security Monitoring & Detection

OPERATIONS

Controls that are not monitored decay silently. Every control in this document emits a signal, and every signal below has a defined owner, alert route and severity class.

MONITORED SIGNALS

SIGNAL	DETECTION FOCUS	SEV
Authentication failures	Rate and distribution per account, address and device; credential-stuffing shapes	High
Authorization failures	Denied object and function access, especially repeated denials from one identity	Crit
Suspicious session activity	Impossible travel, device churn, abnormal concurrency, rotated-token reuse	Crit
Prompt injection attempts	Heuristic and classifier hits, override phrasing, forged role markers, decode-and-follow patterns	High
Tool misuse	Unregistered tool names, schema violations, denial spikes, breaker trips, loop patterns	Crit
Cross-user access attempts	Ownership-check failures and identifier substitution across accounts or tenants	Crit
API abuse	Error-rate and shape anomalies, enumeration, scanning signatures, schema-probe traffic	High
Rate-limit violations	Sustained breaches, distributed low-and-slow patterns, ladder escalations	Med
SSRF attempts	Blocked destinations, metadata-endpoint targets, rebinding indicators, redirect abuse	Crit
File-upload anomalies	Signature mismatches, malware hits, decompression aborts, parser crashes and timeouts	High
Data-exfiltration patterns	Volume and velocity of records returned, redaction triggers, extraction-shaped sessions	Crit
AI cost spikes	Token, tool-call and spend anomalies per user and platform-wide against baseline	High
Unusual payout behaviour	New destinations, velocity breaches, refund and chargeback clusters, off-hours requests	Crit
Admin access	Every privileged session and action, with alerting on unexpected actor, time or scope	Crit
Secret exposure attempts	Secret-pattern hits in responses, logs, repositories, images and build artefacts	Crit

LOGGING & ALERTING PRACTICES

- Real-time alerting with a defined on-call owner per class
- Security dashboards for AI, identity, financial and abuse signals
- Tamper-resistant, append-only security and audit logs
- Documented log retention aligned to policy and obligations
- Automatic secret and PII redaction at write time
- Correlation IDs linking chat turn to tool call to backend effect
- Incident severity classification with response expectations
- Alert tuning reviewed so real signals are not lost in noise
- Detection coverage reviewed whenever a control changes
- Log integrity and delivery gaps themselves alert

19 Incident Response & Recovery

OPERATIONS

Incidents are assumed, not hypothetical. The value of this section is speed and consistency under pressure: a fixed sequence, pre-authorised containment actions, and evidence preserved before anything is cleaned up.

- 1 **Detect** — a signal from §18, a user report, a partner notification or a manual finding opens an incident with a severity class, an owner and a timestamped record
- 2 **Contain** — stop the bleeding with pre-authorised actions: revoke sessions and tokens, disable an account or tool, trip a breaker, freeze payouts, block a destination, or activate the AI kill switch (§17); containment does not wait for full understanding
- 3 **Investigate** — reconstruct the sequence from correlation IDs, audit records and tool logs; establish entry point, identities involved, data touched, and whether the control that should have stopped it was absent, misconfigured or bypassed
- 4 **Eradicate** — remove the cause rather than the symptom: patch the flaw, rotate exposed secrets (§12), delete poisoned memory or knowledge-base content (§6, §8), remove attacker-created state, and close the path that permitted it
- 5 **Recover** — restore service deliberately, verifying that controls are enforced before capability is re-enabled; reverse unauthorised state changes through backend processes with their own authorisation and audit trail
- 6 **Review** — blameless post-incident review covering timeline, detection lag, containment lag, blast radius, customer impact, and notification obligations
- 7 **Improve** — every incident closes with concrete changes: a new or corrected control, a new detection rule, a new regression test in §20, and updated documentation; an incident is not closed until these are tracked

Evidence Rule Preserve before you repair. Logs, payloads, session records and affected state are captured and stored immutably at containment time, because the same data cannot be recovered after eradication.

Standing Readiness

Named on-call owner per severity class, documented escalation path with contact routes tested regularly, pre-approved containment authority so responders never wait for a decision, a communications template for users and partners, and legal and regulatory notification paths identified in advance. Playbooks are rehearsed, not read for the first time during an incident.

19 Incident Response & Recovery

OPERATIONS

CONTINUED — PLAYBOOKS

Each playbook names the first containment action, which is pre-authorised and may be taken immediately, and the follow-through that must complete before the incident is closed.

INCIDENT TYPE	FIRST CONTAINMENT ACTION	FOLLOW-THROUGH
Account compromise	Revoke all sessions and refresh tokens; force credential reset; require step-up on return	Review actions taken while compromised; reverse unauthorised state
Session hijacking or token theft	Invalidate the session family; block the device fingerprint; alert the account owner	Trace reuse detection; verify rotation and cookie controls (§2)
Prompt injection incident	Quarantine the offending content source; tighten the affected input path	Add the vector to the injection test corpus (§20)
Cross-user data exposure	Disable the affected endpoint or tool; scope the affected identities	Fix the ownership check; add an object-level authorization test
Data leak or exfiltration	Cut the egress path; throttle or suspend the actor; enable maximum redaction	Quantify records exposed; assess notification obligations
Unauthorised payout or refund	Freeze payouts platform-wide; halt the queue; contact the payment provider	Recovery attempt; verify no AI path influenced approval (§13)
Payment fraud pattern	Suspend the instrument and destination; raise velocity thresholds to strict	Chargeback handling; tune fraud rules and monitoring
Secret or key exposure	Rotate immediately; revoke the old credential; audit its use window	Find the leak path; add a scanning rule to prevent recurrence
Database compromise or injection	Rotate database credentials; restrict the role; take the affected path offline	Integrity check against backups; convert the query to parameterised form
Admin account compromise	Disable the admin identity; revoke admin sessions; require two-person restore	Full review of every privileged action taken (§22)
Tool or agent misuse	Revoke the tool grant; trip the breaker; suspend the session	Narrow the tool contract; strengthen argument and business validation
Malicious file upload	Quarantine the object; block the uploader; halt downstream processing	Rescan stored objects; verify sandbox and parser limits (§14)
Automated abuse or scraping	Apply strict rate limits and challenges; block the pattern at the edge	Assess data already collected; refine bot detection (§16)
Denial of service or cost attack	Shed load per §17 degradation order; enforce ceilings; use the kill switch if required	Reconcile spend; adjust ceilings and anomaly thresholds

Cross-Cutting Requirement

Every playbook keeps security controls enforced during response. Containment may reduce functionality, disable features or refuse requests, but it never disables authentication, authorization, validation or logging to restore service faster.

20

Continuous Security Testing

ASSURANCE

Controls are claims until they are tested. Each class below is exercised against a non-production environment on a defined cadence, and every fixed finding becomes a permanent regression test.

REQUIRED TEST CLASSES

- ☐ Authentication bypass testing
- ☐ Session security and fixation testing
- ☐ Token replay and rotation-reuse testing
- ☐ Object-level authorization (IDOR) testing
- ☐ Function-level authorization testing
- ☐ Privilege-escalation testing
- ☐ Tenant and cross-user isolation testing
- ☐ Prompt-injection corpus testing (§4)
- ☐ Jailbreak and role-impersonation testing
- ☐ Encoded and multilingual injection testing
- ☐ Tool-abuse and argument-tampering testing
- ☐ Data-exfiltration and extraction testing
- ☐ RAG and memory poisoning testing
- ☐ AI output validation and rendering testing
- ☐ Input validation and API fuzzing
- ☐ Injection testing (SQL, NoSQL, command, template)
- ☐ XSS and CSRF testing
- ☐ SSRF and egress-control testing
- ☐ File and image upload testing
- ☐ Rate-limit, bot and abuse testing
- ☐ Payment and payout flow testing
- ☐ Secrets and dependency scanning (§21)
- ☐ Penetration testing of the full chat path
- ☐ Regression tests for every past finding

CADENCE

ACTIVITY	TRIGGER	FREQUENCY
Automated security test suite	Every pull request and every deployment	Continuous
Dependency and secret scanning	Commit, build and scheduled sweep of the main branch	Continuous + daily
AI adversarial red-team pass	Prompt, tool, model or provider change	Per change + monthly
Authorization matrix review	New endpoint, tool, role or data object	Per change
Full penetration test	Major release or architectural change	Periodic
Control and threat-model review	New feature area or incident outcome	Periodic

Testing Rule

Security testing must continue after launch. Passing a pre-launch test does not prove future security.

21 AI & Software Supply-Chain Security

NEW IN V4.0

The chatbot inherits the security posture of everything it depends on: packages, containers, build pipelines, model providers and third-party integrations. Each dependency is treated as an untrusted component with a known version and a known owner.

CONTROL	REQUIREMENT	LEVEL
Pinned versions	Exact versions only; no floating ranges or mutable tags in production builds	MUST
Committed lockfiles	Lockfiles are committed, reviewed and verified during build; integrity hashes enforced	MUST
Vulnerability scanning	Automated scanning of dependencies on every build with severity-based blocking	MUST
Software bill of materials	SBOM generated per release and retained so exposure can be answered quickly	SHOULD
Dependency review	New packages reviewed for maintenance, ownership, permissions and typosquatting before adoption	MUST
Container image scanning	Images scanned before promotion; unscanned images never reach production	MUST
Base image currency	Documented policy for rebuilding on base-image and runtime security updates	MUST
AI provider review	Provider assessed for data handling, retention, training use, isolation and incident process	MUST
Model change review	Model or version changes re-run the AI red-team and injection suites before rollout	MUST
Integration review	Third-party integrations reviewed for scope, egress, secrets and failure behaviour	MUST
Repository secret scanning	Pre-commit and server-side scanning; any hit triggers rotation, not just removal	MUST
Signed builds	Artefacts signed and verified at deploy; provenance recorded for each release	SHOULD
CI/CD access control	Least-privilege pipeline identities, protected branches, reviewed workflow changes, scoped short-lived deploy credentials	MUST
Build isolation	Builds run without production data or production secrets; no untrusted code executes with release privileges	MUST

AI-Specific Supply-Chain Risk

Prompts, tool definitions, system instructions, retrieval sources and model configuration are versioned, reviewed and deployed like code — never edited live in production. A prompt or tool-contract change is a security change and passes the same review and test gates as application code.

22 Administrative Access Security

PRIVILEGED

Administrative capability is the highest-value target on the platform and is therefore kept completely outside the chatbot's reach. No conversational path, however phrased, leads to a privileged action.

CONTROL	REQUIREMENT	LEVEL
Separate admin interface	Administration lives on its own surface with its own authentication, not behind a flag in the user application	MUST
Mandatory MFA	Multi-factor authentication required for every administrative identity, with no exemptions	MUST
Separate admin sessions	Admin sessions are distinct from user sessions and cannot be derived from one; no shared cookie scope	MUST
Shorter timeouts	Reduced idle and absolute lifetimes relative to user sessions	MUST
Re-authentication	Fresh authentication required for critical actions regardless of session age	MUST
Least privilege	Granular permissions per function; no blanket super-admin for routine work	MUST
Role separation	Support, finance, moderation and engineering roles are distinct with non-overlapping sensitive powers	MUST
Complete audit logging	Every privileged view and action recorded with actor, target, before-and-after state, time and reason	MUST
Two-person approval	High-risk actions — large payouts, bulk changes, permission grants, data exports, deletions — require a second authorised approver	MUST
Network restriction	Admin surfaces restricted by network or device posture where operationally feasible	SHOULD
Access review	Periodic review of administrative accounts and permissions; prompt revocation on role change or departure	MUST

PROHIBITED BY DESIGN

- ✗ No administrative access through the chatbot, in any conversation, for any user
- ✗ No AI-created, AI-granted or AI-modified administrator privileges
- ✗ No AI-controlled administrator actions, including on behalf of a verified administrator
- ✗ No administrative tool exposed to the AI runtime, even read-only, even behind confirmation
- ✗ No privilege inheritance from an administrator's chat session into the AI's tool grants

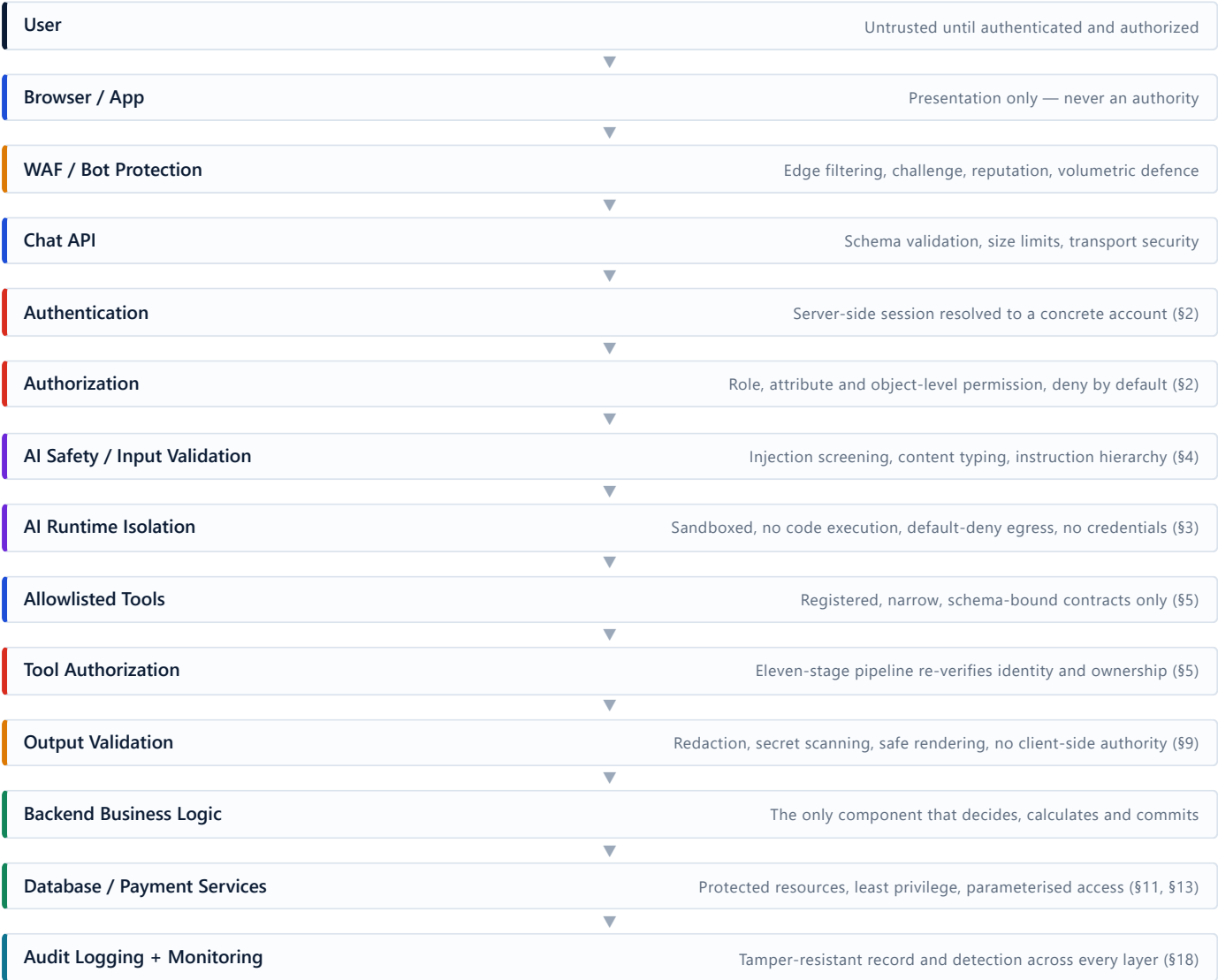
Boundary An administrator using the chatbot is still a user. Their questions may be answered; their privileges are never exercised through it.

23

Final Security Architecture

REFERENCE FLOW

Every sensitive request traverses the full path below. No layer may be skipped, reordered or bypassed, and any layer that cannot complete its check fails the request closed (§1).



Architecture Boundary AI may interpret, explain and recommend. Only trusted backend services may authenticate, authorize, calculate, modify state or commit sensitive operations.

24

Production Release Gate

SIGN-OFF

Each check is verified by evidence — a passing test, a reviewed configuration or a demonstrated behaviour — not by assertion. Every row is blocking.

#	CHECK	VERIFICATION EVIDENCE	GATE
1	Authentication	Server-side sessions enforced; bypass tests pass (§2)	BLOCKING
2	Session security	Rotation, revocation, timeouts, secure cookies verified (§2)	BLOCKING
3	Function-level authorization	Deny-by-default matrix reviewed per endpoint and tool (§2)	BLOCKING
4	Object-level authorization	IDOR and cross-user tests pass on every data path (§2)	BLOCKING
5	AI runtime isolation	No code execution, no credentials, default-deny egress (§3)	BLOCKING
6	Prompt-injection defence	Full vector corpus executed; instruction hierarchy holds (§4)	BLOCKING
7	Tool-chain security	Eleven-stage pipeline enforced; allowlist and limits verified (§5)	BLOCKING
8	Memory & context security	Scoping, expiry and poisoning resistance tested (§6)	BLOCKING
9	Exfiltration prevention	Volume limits, redaction and egress controls demonstrated (§7)	BLOCKING
10	RAG & knowledge base	Authorization-aware retrieval; source vetting in place (§8)	BLOCKING
11	AI output security	Output treated as data; safe rendering; secret scanning (§9)	BLOCKING
12	API & infrastructure	Transport, headers, CORS, validation and limits confirmed (§10)	BLOCKING
13	Database security	Parameterised access, least-privilege roles, scoped queries (§11)	BLOCKING
14	Secrets & keys	No secrets in code, prompts, client or logs; rotation ready (§12)	BLOCKING
15	Payment & financial	Backend-only authorisation, webhook verification, velocity checks (§13)	BLOCKING
16	File & image security	Type, size, signature, malware and parser limits enforced (§14)	BLOCKING
17	Web & SSRF security	Destination allowlist, redirect and rebinding defence verified (§15)	BLOCKING
18	Abuse & automation defence	Layered rate limits, bot controls and lockouts active (§16)	BLOCKING
19	AI cost & DoS protection	Ceilings, breakers, kill switch and degradation order tested (§17)	BLOCKING
20	Monitoring & incident readiness	Signals alerting to a named owner; playbooks rehearsed (§18, §19)	BLOCKING

STANDING GATES — VERIFIED BEFORE AND AFTER RELEASE

- ☐ Continuous security testing operating on cadence (§20)
- ☐ Supply-chain controls enforced in the pipeline (§21)
- ☐ Administrative access separated from the chatbot (§22)
- ☐ Fail-closed behaviour confirmed for every check above (§1)

25

Final Security Principle

DOCTRINE

Security Principle SaveHatke security is defense in depth. The AI is never the final authority. The frontend is never the final authority. The user is never trusted solely on claims. The backend independently authenticates, authorizes, validates and commits every sensitive operation. If a security check cannot establish that an operation is permitted, the operation fails closed. Security controls must be continuously tested, monitored, reviewed and updated as the platform evolves.

THE MODEL IN FOUR LINES

COMPONENT	ROLE IN THE SECURITY MODEL
AI	Assistant. Interprets, explains and recommends. Holds no credentials, no privileges and no decision authority.
Backend	Authority. The only component that authenticates, authorizes, calculates, validates and commits state.
Database	Protected resource. Reached only through least-privilege, parameterised, ownership-scoped backend access.
User	Untrusted until authenticated and authorized. Claims about identity, role, balance or entitlement are never accepted as fact.

SCOPE OF THIS BASELINE

What This Document Claims — And What It Does Not

This baseline specifies the strongest practical security controls for the SaveHatke AI chatbot and its supporting services. It does not claim the platform is impossible to attack, and no control set can offer that. It claims that sensitive operations are independently verified by trusted backend services, that failures resolve closed rather than open, that the AI is structurally prevented from acting as an authority, and that the resulting posture is continuously tested, monitored and improved as threats and the platform change.

REVIEW & SIGN-OFF

RESPONSIBILITY	OWNER	CONFIRMATION REQUIRED
Security review		All §24 gate checks evidenced and blocking failures cleared
Engineering review		Controls implemented as specified; no bypass paths in code
AI & tool review		Prompts, tool contracts and limits versioned and tested
Operations readiness		Monitoring live, on-call assigned, playbooks rehearsed
Production approval		Release authorised with date, version and evidence reference

SH-SEC-CHATBOT-4.0 · Supersedes v3.2 · This baseline is a living document: it is reviewed after every security incident, before every major release, and on a recurring schedule. Controls are added and strengthened as the platform evolves; existing controls are not removed or weakened without a documented security review.